Ejemplo sobre el uso de pila en Assembler

Arquitectura del Computador - LCC - FCEIA-UNR

2 de noviembre de 2026

Este ejemplo tiene como objetivo ilustrar algunos aspectos del uso de la pila. Asimismo, permite familiarizarse con el uso de GDB para examinar el contenido de la pila durante la ejecución del programa.

Dado el siguiente código en Assembler x86-64 en el archivo `stack_1.s`:

```
.text
.global main

main:
    movq $10, %rax
    movq $20, %r10

    pushq %rax        # Apila el valor 10
    pushq %r10        # Apila el valor 20

    popq %rcx         # Desapila 20 en rcx
    popq %rdx         # Desapila 10 en rdx

    xorl %eax, %eax   # Retorna 0
    ret
```

Compilar utilizando la opción `-g` para poder *debuggear* utilizando GDB:

```
$ gcc -g stack_1.s
```

Luego ejecutar utilizando GDB:

```
gdb ./a.out
```

Una vez que estamos dentro de la sesión de *debugging*, ponemos un *breakpoint* en `main` y ejecutamos el comando `run`:

```
(gdb) br main
Breakpoint 1, main () at main.S:5
(gdb) r
Starting program: /home/a.out

5           movq $10, %rax
```

Primero vemos el estado del registro `rsp`:

```
(gdb) i r rsp
rsp            0x7fffffffeb88      0x7fffffffeb88
```

Luego vamos ejecutando instrucción a instrucción utilizando el comando `next` hasta ejecutar las dos instrucciones `pushq` y volvemos a chequear el estado de registro `rsp`:

```
(gdb) i r rsp
rsp            0x7fffffffeb78      0x7fffffffeb78
```

Efectivamente, corroboramos que el valor de `rsp` disminuyó en 16 (8 por cada instrucción `rsp`).

Además, podemos visualizar los dos valores almacenados en la pila:

```
(gdb) x/2gd $rsp
0x7fffffffeb78: 20      10
```

Observamos que el valor 20, que fue el último en apilarse, se encuentra en la dirección señalada por `rsp`, mientras que el valor 10 se encuentra 8 bytes hacia arriba. Esto ilustra el funcionamiento LIFO (*Last In, First Out*) de la pila: el último valor apilado es el primero que se recupera. También sería interesante ejecutar:

```
(gdb) p/x $rsp
```

antes y después de cada pushq y popq, para comprobar cómo cambia el puntero de pila, o directamente `display $rsp` para que nos muestre el valor luego de cada instrucción.

Finalmente, ejecutar las dos instrucciones `popq` y volver a chequear el valor del registro `rsp` y los registros `rcx` y `rdx`:

```
(gdb) i r rsp rcx rdx
rsp            0x7fffffffeb88      0x7fffffffeb88
rcx            0x14                20
rdx            0xa                 10
```

Comprobamos que los valores fueron recuperados en orden inverso al que se apilaron: primero el valor 20, almacenado en `rcx`, y luego el valor 10, almacenado en `rdx`.

Además, el registro `rsp` recuperó su valor inicial, lo que indica que los dos elementos fueron retirados de la pila. Es importante destacar que las instrucciones `popq` no borran los datos de la memoria, sino que recuperan sus valores e incrementan el puntero de pila.

La siguiente figura ilustra la evolución de la pila durante la ejecución del código anterior. En particular, se muestra cómo cambia el valor del registro `rsp` después de cada instrucción.

